VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
YASHASVI KESARWANI	24BCE10647
Slot	**Date of Submission**
D11	31/03/26

1. Introduction

The Open Source Audit takes a close look at how free and open-source software (FOSS) actually works in practice—both in terms of its technical design and the ideas behind it. Instead of sticking to a general definition of “open source,” this project focuses on one specific tool that has had a major impact on modern computing. By studying Git, I want to understand not only how it operates within a Linux system, but also how the GPL v2 license and the broader values of the open-source community have influenced the way it was built and adopted.

I chose Git because it stands out as one of the most successful examples of open-source software. It didn’t come from a big company or a startup with funding—it emerged during a difficult moment in the Linux kernel community, when access to a key proprietary tool was suddenly taken away. Rather than being set back, the community responded by creating something better and more open. That story highlights the strength of open source: when limitations are imposed, people can come together to build solutions that remain accessible to everyone. This mix of technical innovation and philosophy makes Git an especially interesting subject to study.

The report is organised to reflect both sides of open source. It starts with the background and ideas that led to Git’s creation, including the BitKeeper situation and the role of the GPL v2 license. From there, it moves into how Git actually functions on a Linux system, followed by a look at the tools and platforms that support it. It also includes a comparison with proprietary alternatives like Perforce, and ends with a set of five shell scripts I created to automate different auditing tasks.

On a personal level, this project marks an important step in my development as a programmer. As I move toward a professional career, understanding software licenses isn’t just about following rules—it’s about recognising the responsibilities that come with using other people’s work. The tools I depend on every day exist because others chose to share what they built. Studying Git has helped me start to see how this shared effort works, and how I might contribute to it in the future.

Overall, this audit follows Git’s path from a short development effort by Linus Torvalds in 2005 to the massive ecosystem it supports today.

2. Choosing your software

Your choice	Git

Part A — Origin and Philosophy (Units 1 & 2)

A1 The Origin

Git is another example of the concept that Necessity is the Mother of Innovation and Invention. It was the result of Bitkeeper access being revoked in 2005 that Linus Torvalds, the Finnish American who created the Linux kernel in 1991, created a version control system namely Git. He not only substituted BitKeeper, he made something far more than that. He revolutionised version control to a very different level for the many upcoming years. Git is still used, in fact it powers other famous tools like GitHub and GitLab.

While developing open source software a version control software is needed to manage the all the data being written by a number of developers from all over the world. They are required to monitor the changes in different sections of the code made by these different people.

The various softwares used before included :

- Concurrent Version Systems
- Subversion
- BitKeeper

Problems associated with these softwares :

- They all were centralised systems meaning they were depended on one central server or device.
- This greatly reduced the speed and efficiency of these softwares.
- Ever after this there were many problems with branching and merging.
- Even if you were able to mange this there was always the concern of network's speed and availability.
- Lastly, we can't forget that some of these softwares were not open source but proprietary softwares.

Linux Kernel :

While talking about the open source projects we can't forget one of the most vast ones which was the Linux Kernel. As of today it is maintained

by over a thousand brilliant developers from across the nations headed by it's creator Mr Linus Torvalds. For the initial duration of the project they used to manual patch sharing via emails as their version control system. They later shifted to Bitkeeper as their Version Control System. It was a proprietary software which gave licenses to it's users.

The Problem that led to Git :

Earlier they developers were enjoying the benefits of Bitkeeper without because of their deal with the Bitkeeper. But, around the year 2005 due to the disputes between the version control company and our open source Linux community of developers the deal was called off.

At that time Linux has reached a level where they had a very huge codebase but had lost their version control system. They managed it without any proper tool but it was becoming impossible to sustain it even for a few months. Although, Linus reached and evaluated other ver control systems they were just too slow for the development of kernel.

The Solution :

To get out of this misery he wanted a tool as soon as possible but he didn't want a newer version of Bitkeeper he wanted something far more powerful and better than that. So he decided to make it on his own. He wanted to build a system with the following parameters :

- **Speed :**
  - Handle large codebases.
  - Fast commit, diff and merge.
- **Data Integrity** :
  - Secure it cryptographically.
  - Changes are tracked using hashing.
- **Distributed System :**
  - Follow distributed architecture.
  - Offline functionality.
- **Advanced Branching and Merging Mechanisms**
  - Rollback
  - Can merge readily.

- **Simple and Crisp**
  - Uses Snapshots.

The Result :

It was not less of a miracle that Git was created in just a span of a few weeks. It was later improved and made a more advanced version of itself. It became even more user friendly and more reliable and was widely adopted by various organisations due to its core features.

As of today if you think of any version control system Git is the only one talk about. It has become an industry standard for our generation. It is also used in powerful platforms like GitHub and GitLab.

A2 License

Among the various open source license options, Git has been licensed under General Public License v2 also known as GPL v2.

The Four Freedoms of free software :

1. **Freedom to use :**
   You can use it for any purpose be it commercial or personal.

2. **Freedom to study :**
   No restriction on the source code.

3. **Freedom to modify :**
   You can even modify the source code as you want.

4. **Freedom to distribute :**
   You can share your own version. You can even privatise it.

Git provides all the these four features which are also the reason for Git’s widespread and standardised use by developers. The GPL v2 mentioned earlier ensures that Git’s users are never restricted of the source code.

Redistribution :

Any company, organisation or community can take it modify it and even sell it as part of their product with one condition that is they have to make the source code public. It will also have to be distributed under the same license as Git which is GPL v2.

Git follows the Copyleft principle which means any modification of the inherited code also has tot be made public to all the users.

GPL VS MIT :

GPL
- Strictly follows the principles of open source softwares.
- It is mandatory to share the any modifications publicly.

MIT
- Unlike GPL it allows the use of software as a proprietary one.
- No compulsion on sharing the software.

Choosing between the two is totally based on the requirement and the situation of the software and the developer.
Personally for monetary benefits I would most probably use MIT to share my intellectual property while earning a living from it.

FREE :
Free as in free Beer means something which doesn't require any monetary transaction or for any kind of transaction while Free in Freedom means your control over the software.

You can have a free software but you can't modify it's source code.

A3 Ethics
Should all software be open source :

For :
1. It will help a lot in the advancement of the technology.
2. Make system more transparent, making it free form malicious intentions.
3. Increase Security.
4. It will support Sharing is Caring.

Against:
1. Increases financial problems for the developers.
2. The number of people investing in this would be significantly reduced because of no payout.
3. Reduced RND by companies.
4. It will basically disturb the whole phenomenon of healthy competition.

Is it ethical for companies to profit from open source?

Frankly speaking if you have put you time and effort into improving an open source software there is it is not wrong to charge for your work. It depends on the people if they want to buy it from you or not.
But if you are being forced to make it free for all the determination for doing the task is reduced a lot as the rear is removed from the equation.

“Standing on the shoulders of Giants” :

Most of the time new creations are based on the principles of another creation or they are the extension of the older creation.
In respect to the developer’s world this is the scenario when he or she uses the core libraries to make a new more advanced and more powerful one. It’s not only the case for Git or open source softwares but it is true for any software.
Open Source reduce the redundancy by applying this principle. We could argue about the originality but that is a very small concern against the benefits it gives for the World.

Part B — Linux Footprint (Unit 2)

Installation Method :

The most common way to install it is by using a package manger.
I have done it on my Arch Linux as a result I have used paceman.

Sudo paceman -S git

And that’s it for the installation part. Now we can use it.

Key Directories :

The main executable binary of Git is /usr/bin/git .
Git handles its configuration in a few different layers. At the system level, there’s a file located at `/etc/gitconfig`, which sets defaults for everyone using the machine. Then each user has their own config file in their home directory, `~/.gitconfig`, where things like their name and email are stored. On top of that, every individual repository has its own settings inside `.git/config`. This setup makes Git really flexible, since you can define settings globally, for a specific user, or just for a single project.

When it comes to how Git runs, it’s pretty simple compared to a lot of other tools. It doesn’t run in the background or as a service. Instead, it just runs under whoever is currently logged in. You can check your

current user with a command like `whoami`, and that's exactly the level of access Git will have. There's no special Git user or group on the system. This approach is actually safer, because Git can only access files that the user already has permission to use, instead of running with elevated privileges.

Because of this, Git isn't something you can start or stop like a service. Commands like `systemctl start git` won't work, since there's no daemon running in the background. Git is just a command-line tool—you run a command, it does its job, and then it exits. That said, Git is often used alongside other services like SSH or web servers when working with remote repositories, but those services are managed separately.

As for updates, Git is maintained by a large open-source community, and new versions come out regularly with improvements, bug fixes, and security patches. On most Linux systems, updating Git is handled through the package manager. For example, you might run `sudo apt update` followed by `sudo apt upgrade git` to get the latest version. Unlike proprietary software, where updates depend on a single company, Git benefits from contributions from developers all over the world, which helps issues get spotted and fixed more quickly.

Part C — The FOSS Ecosystem (Units 3 & 4)

It's a mistake to think of Git as just a tool you run in your terminal. In reality, it's a central pillar in a massive, interconnected ecosystem of open-source software. No piece of software is an island, and Git is a perfect example of how different projects lean on each other to actually get things done.

Standing on the Shoulders of Giants :

Git doesn't try to reinvent the wheel. Instead, it relies on rock-solid libraries that have been around for decades. For example, it uses zlib for compression and OpenSSL to keep communications secure. It's usually compiled with GCC, another open-source staple. This isn't just a technical detail; it's a philosophy. By reusing these existing tools, Git stays lean and benefits from the security and stability that the broader community has already built.

More Than Just a Command Line :

While Git is the "engine" under the hood, most people experience it through platforms like GitHub or GitLab. These sites take Git's core version control and wrap it in a layer of social features—things like pull

requests, issue trackers, and CI/CD pipelines. Even if you spend all day in a web interface, Git is still doing the heavy lifting behind the scenes.

Where Git Fits in Development :
You'll often see Git used alongside the "LAMP" stack (Linux, Apache, MySQL, PHP). It isn't officially part of that stack, but honestly, it's hard to imagine modern web dev without it. It's the invisible thread that keeps a team's code synced up and prevents everyone from accidentally overwriting each other's work.

A Community-Driven Evolution :
Finally, there's the human element. Even though Linus Torvalds started the project, it's been maintained by Junio Hamano and a massive global community for years. This isn't a corporate product; it's a living project shaped by mailing list debates and real-world needs. That community-driven model is exactly why Git has remained the industry standard for so long—it evolves because the people using it are the ones building it.

Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	Open Source	Proprietary Alternative
Cost	Free as licensed under GNU GPL v2.	There might be free trials but you can't use them for large projects. So you will have to pay for the software.
Security (who can audit the code?)	The source code is publicly available, so anyone can re-view it and find vulnerabili-ties.	The source code is closed, so only the company can check and fix security issues.
Support and reliability	Supported by a large global community with extensive documentation, forums, and frequent updates	Provides official customer support, but depends on sub-scription and company re-sponse time.

Dimension	Open Source	Proprietary Alternative
Freedom to modify	Users can freely modify Git and even create their own versions according to their needs.	Users cannot modify the soft-ware and must follow com-pany-imposed restrictions.
Community vs corporate control	Developed and maintained by a global open-source community where contribu-tions are shared	Controlled by a company, where decisions are driven by business goals.
Your overall verdict	Git is an interconnected open-source project that both leans on existing tools and forms the cornerstone of other development platforms.	t demonstrates the key open-source principle where each project strengthens others.

Conclusion :
I can’t even imagine what the world of version control be without Git. It has helped the developers to make large projects without any trouble in managing with various people. It has made collaboration with other team members very easy. It has made managing the large codebase like a basic task. We all know that it has become the industry standard as a result of that.

3. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

```
#!/bin/bash
# Script 1: System Info Summary
# Author: Yashasvi Kesarwani

NAME=“Yashasvi Kesarwani"
TOOL_USED=“Git"

KERNEL_VER=$(uname -r)
CURRENT_USER=$(whoami)
CURRENT_TIME=$(date "+%d-%m-%Y %H:%M:%S”)

# Uptime check
if command -v uptime >/dev/null 2>&1; then
        SYS_UPTIME=$(uptime -p)
else
        SYS_UPTIME="Unavailable"
fi

# OS Detection
if [ -f /etc/os-release ]; then
        OS_NAME=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d "')
else
        OS_NAME="Unknown / Git Bash"
fi

echo "--------------------------------"
echo " SYSTEM REPORT DASHBOARD"
echo "--------------------------------"
echo "Student Name : $NAME"
echo "Tool Used : $TOOL_USED"
echo "User : $CURRENT_USER"
echo "Kernel Ver : $KERNEL_VER"
echo "Operating OS : $OS_NAME"
echo "System Uptime: $SYS_UPTIME"
echo "Timestamp : $CURRENT_TIME"
echo "License Info : GPL (Linux)"
echo “————————————————————"
```

```
[root@alarm oss-project]# ./script1b.sh
========================================
 Open Source Audit – Yashasvi Kesarwani
========================================
Software: Git echo Distro : Arch Linux ARM
Kernel : 6.19.0-2-aarch64-ARCH echo User : root
Home : /root
Uptime : up 22 minutes
Date : Tue Mar 31 16:05:47 UTC 2026

License : This Linux system is typically licensed under GPL (Free Software License)
========================================
```

Displays system information such as kernel version, current user, uptime, and Linux distribution. Demonstrates variables and command substitution.

Script 2 — FOSS Package Inspector (Unit 2)

```
#!/bin/bash
# Script 2: Software Checker
# Author: Yashasvi Kesarwani

# Check if package is installed
if pacman -Q $PACKAGE &>/dev/null; then
echo "$PACKAGE is installed."
pacman -Qi $PACKAGE | grep -E 'Version|License|Description'
else
echo "$PACKAGE is NOT installed."
fi

# Case statement
case $PACKAGE in
git) echo "Git: distributed version control system powering modern development" ;;
apache) echo "Apache: web server powering the internet" ;;
mysql) echo "MySQL: database for web applications" ;;
vlc) echo "VLC: media player supporting all formats" ;;
*) echo "Unknown
esac
```

```
[root@alarm oss-project]# ./script2a.sh
Git: distributed version control system powering modern development
```

Checks whether a package (Git) is installed and displays its details. Uses conditional statements and a case structure.

Script 3 — Disk and Permission Auditor (Unit 2)

```
#!/bin/bash
# Script 3: Directory & Permission Checker
# Author: Yashasvi Kesarwani

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done

# Check Git config file
echo ""
echo "Checking Git configuration:"
if [ -f ~/.gitconfig ]; then
    echo "Git config found at ~/.gitconfig"
    ls -l ~/.gitconfig
else
    echo "Git config not found"
fi
```

```
[root@alarm oss-project]# ./script3a.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 8.3M
/var/log => Permissions: drwxr-xr-x root root | Size: 389M
/home => Permissions: drwxr-xr-x root root | Size: 1.8M
/usr/bin => Permissions: drwxr-xr-x root root | Size: 212M
/tmp => Permissions: drwxrwxrwt root root | Size: 4.0K
```

Analyzes important system directories and reports their size, permissions, and ownership. Uses loops and text processing tools.

Script 4 — Log File Analyser (Units 2 & 5)

```
#!/bin/bash
# Script 4: Log Keyword Scanner
# Author: Yashasvi Kesarwani
# Usage: ./script4.sh /var/log/pacman.log error

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
   echo "Error: File $LOGFILE not found."
   exit 1
fi

# Count keyword occurrences
while IFS= read -r LINE; do
   if echo "$LINE" | grep -iq "$KEYWORD"; then
      COUNT=$((COUNT + 1))
   fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# Show last 5 matches
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

```
[root@alarm oss-project]# ./script4.sh
Error: File  not found.
[root@alarm oss-project]# ./script4.sh /var/log/pacman.log error
Keyword 'error' found 10 times in /var/log/pacman.log

Last 5 matching lines:
[2026-02-20T09:33:54+0000] [ALPM-SCRIPTLET] ==> ERROR: file not found: '/etc/vconsole.conf'
[2026-02-20T09:33:59+0000] [ALPM-SCRIPTLET] bsdtar: Write error
[2026-02-20T09:33:59+0000] [ALPM-SCRIPTLET] bsdtar: Write error
[2026-02-20T09:33:59+0000] [ALPM-SCRIPTLET] ==> ERROR: Initcpio image generation FAILED: 'sort' reported an error
[2026-03-31T15:12:22+0000] [ALPM] installed perl-error (0.17030-3)
```

Reads a log file and counts occurrences of a keyword. Displays the number of matches and recent log entries.

Script 5 — The Open Source Manifesto Generator (Unit 5)

```
#!/bin/bash
# Script 5: Personal Open Source Note
# Author: Yashasvi Kesarwani

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Create manifesto
echo "On $DATE, I believe that open source is about $FREEDOM." > $OUTPUT
echo "I use tools like $TOOL every day, and I aspire to build $BUILD and share it freely with others." >>
$OUTPUT

echo ""
echo "Manifesto saved to $OUTPUT"
echo "--------------------------------"
cat $OUTPUT
```

```
[root@alarm oss-project]# ./script5.sh
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Freedom
3. Name one thing you would build and share freely: An app
Manifesto saved to manifesto_root.txt echo --------------------------------
On 31 March 2026, I believe that open source is about Freedom.
I use tools like Git every day, and I aspire to build An app and share it freely with others.
```

Generates a personalized manifesto by taking user input and saving it to a file. Demonstrates user input and file handling.